



PROJETO PRÁTICO

Desafio Final de QA

Automação de Testes: UI, API e CI/CD

Cypress · Robot Framework · Testes de API · GitHub Actions —
com cenários em BDD para testes manuais e gravação de evidências.

Instrutor: Douglas Adriano Queiroz

O que veremos nesta aula



O desafio

3 repositórios, 1 semana de prazo



Parte 1 — Interface (UI)

SauceDemo com Cypress e Robot



Parte 2 — API

JSONPlaceholder: GET e POST



Parte 3 — CI/CD

Pipelines no GitHub Actions



Cenários em BDD

Gherkin para testes manuais



Gravação de evidências

Como registrar suas execuções

O desafio em resumo

Chegou a hora de mostrar tudo o que você aprendeu no curso: um desafio real de processo seletivo de QA.

Missão: criar 3 projetos de testes automatizados, cada um em um repositório público no GitHub.

Prazo: 1 semana a partir do recebimento do desafio.

Alvos: SauceDemo (e-commerce fictício, sem API) e JSONPlaceholder (API pública fake).

Extra desta aula: cenários BDD para executar os testes manualmente e gravar as evidências antes de automatizar.



Pense como uma prova de direção

Você já fez as aulas teóricas (CTFL) e as práticas. Agora o examinador quer ver você dirigindo sozinho: planejar, testar, automatizar e provar que funciona.



Leve daqui: Leia o desafio inteiro antes de codar. Entender o que será avaliado é o primeiro teste.

Testes de Interface no SauceDemo



Dois fluxos, duas ferramentas

Fluxo 1 — Login: entrar no sistema com usuário e senha válidos (e cenários de erro).

Fluxo 2 — Checkout: adicionar produto ao carrinho e concluir a compra até o “Thank you for your order!”.

Cypress: os dois fluxos automatizados em JavaScript.

Robot Framework: os mesmos fluxos, escritos no formato BDD (é isso que será avaliado!).

ACESSO AO SAUCEDEMO

URL: <https://www.saucedemo.com>

Usuário: `standard_user`

Senha: `secret_sauce`

Outros usuários para explorar:
`locked_out_user`, `problem_user`,
`performance_glitch_user`



Vídeo de referência do fluxo: youtu.be/T68oJWTKoIA
— assista antes de escrever os cenários.

BDD e Gherkin: testes que contam histórias

BDD (Behavior-Driven Development) descreve o comportamento do sistema em linguagem que todo mundo entende — como uma receita de bolo: qualquer pessoa consegue seguir os passos e conferir o resultado.

Dado — o contexto: onde a história começa.

Quando — a ação: o que o usuário faz.

Então — o resultado esperado: o que deve acontecer.

ANATOMIA DE UM CENÁRIO

Funcionalidade: o que estamos testando

Cenário: a situação específica

Dado um contexto inicial

Quando o usuário faz uma ação

E faz outra ação

Então o resultado esperado acontece



Leve daqui: Cenário BDD bom é aquele que um colega executa manualmente sem te perguntar nada. Escreva primeiro, automatize depois — o mesmo texto vira seu roteiro de teste manual, de gravação e de automação no Robot.

Fluxo 1: Login

CENÁRIO 1 — CAMINHO FELIZ

Funcionalidade: Login no SauceDemo

Cenário: Login com credenciais válidas

Dado que estou na página de login

Quando preencho o usuário "standard_user"

E preencho a senha "secret_sauce"

E clico no botão "Login"

Então vejo a página de produtos

E o título "Products" está visível

CENÁRIO 2 — SENHA INVÁLIDA

Cenário: Login com senha errada

Dado que estou na página de login

Quando informo "standard_user"

E a senha "senha_errada"

E clico em "Login"

Então vejo o erro "Username and password do not match..."

CENÁRIO 3 — USUÁRIO BLOQUEADO

Cenário: Usuário bloqueado

Dado que estou na página de login

Quando informo "locked_out_user"

E a senha "secret_sauce"

E clico em "Login"

Então vejo "Sorry, this user has been locked out."

Fluxo 2: Checkout

CENÁRIO 1 — COMPRA COMPLETA

Funcionalidade: Checkout no SauceDemo

Cenário: Compra concluída com sucesso

Dado que estou logado como "standard_user"
E adicionei "Sauce Labs Backpack" ao carrinho
Quando abro o carrinho e clico em "Checkout"
E preencho nome "Douglas", sobrenome "QA"
E o CEP "88000-000"
E clico em "Continue" e depois em "Finish"
Então vejo "Thank you for your order!"
E o carrinho fica vazio

CENÁRIO 2 — CAMPO OBRIGATÓRIO

Cenário: Checkout sem o nome

Dado que estou na etapa de informações do checkout
Quando clico em "Continue" sem preencher o nome
Então vejo o erro "Error: First Name is required"



Leve daqui: Valide sempre o resultado final (mensagem, URL, carrinho vazio) — clicar sem conferir não é testar.

Testes de API no JSONPlaceholder

GET — CONSULTAR COMENTÁRIOS

GET `https://jsonplaceholder.typicode.com/comments`

Validar:

- Então status code 200
- E lista de comentários no corpo
- E campos postId, id, name, email e body

POST — CRIAR ÁLBUM

POST `https://jsonplaceholder.typicode.com/albums`

Body: { "userId": 1,
"title": "Meu álbum de testes" }

Validar:

- Então status code 201
- E resposta com "id" e o "title"



Lembra da analogia da casa? A API é a casa, cada endpoint é uma porta e o token é a chave. No JSONPlaceholder as portas estão destrancadas (sem autenticação) — perfeito para treinar. A tecnologia é livre: Postman/Newman, RestAssured, Supertest...

API: GET e POST

CENÁRIO — GET /COMMENTS

Funcionalidade: Consulta de comentários

Cenário: Buscar comentários

Dado que a API está disponível

Quando envio GET para "/comments"

Então o status code é 200

E o corpo tem uma lista de comentários

E cada item tem postId, id, name, email e body

CENÁRIO — POST /ALBUMS

Funcionalidade: Criação de álbuns

Cenário: Criar um novo álbum

Dado que a API está disponível

Quando envio POST para "/albums"

E o body { "userId": 1, "title": "Meu álbum de testes" }

Então o status code é 201

E a resposta contém o novo "id"

E o "title" enviado



Leve daqui: Execute os dois manualmente no Postman (e grave!) antes de automatizar — você já valida a API e o roteiro de uma vez.

Executando os cenários manualmente



O ritual do teste manual

- 1. Prepare:** navegador limpo, cenário BDD aberto ao lado.
- 2. Grave:** inicie a gravação de tela antes do primeiro passo.
- 3. Execute:** siga o Dado/Quando/Então literalmente.
- 4. Registre:** status (passou/falhou) e o arquivo de evidência.
- 5. Reporte:** se falhar, documente como bug (Aula B4).

Planilha de execução (exemplo)

Cenário	Status	Evidência
Login válido	Passou	login_ok.mp4
Senha inválida	Passou	login_erro.mp4
Usuário bloqueado	Passou	login_bloq.mp4
Compra completa	Passou	checkout_ok.mp4
Checkout sem nome	Falhou?	bug_001.mp4
GET /comments	Passou	api_get.mp4
POST /albums	Passou	api_post.mp4

Como gravar a execução dos testes



Windows

Xbox Game Bar:
Win + G (grátis, nativo)



macOS

Gravador nativo:
Cmd + Shift + 5



OBS Studio

Grátis e completo,
para vídeos maiores



Loom / Extensões

Grava e já gera link
para colar no README

Boas práticas de gravação

Roteiro pronto: o cenário BDD é o seu script — um vídeo curto por cenário.

Tela limpa: feche abas e notificações; deixe visível só o que importa.

Nomeie bem: login_sucesso.mp4, checkout_erro_nome.mp4 — nome que se explica sozinho.

Mostre o resultado: termine o vídeo na validação do Então (mensagem, status code).

Onde entregar: suba os vídeos ou cole os links na seção “Evidências” do README.

Automatizados também: grave um run do Cypress/Robot — fica ótimo no portfólio.

GitHub Actions: testes que rodam sozinhos



O que o pipeline precisa fazer

1 workflow por repositório: Robot, Cypress e API — independentes.

Gatilho automático: rodar a cada push e pull request.

Relatórios como artefatos: suba o report (HTML/XML) nos artifacts do run para consulta fácil.

Paralelização: se aplicável, divida os testes para ganhar tempo.

EXEMPLO MÍNIMO — CYPRESS.YML

```
name: Testes Cypress
on: [push, pull_request]
jobs:
  e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npx cypress run
      - uses: actions/upload-artifact@v4
        with:
          name: relatorio
          path: cypress/reports
```

Três repositórios públicos no GitHub



REPOSITÓRIO 1

UI — Robot Framework

Login + Checkout em BDD, com keywords organizadas e tags.



REPOSITÓRIO 2

UI — Cypress

Os mesmos fluxos em specs claras, com comandos customizados.



REPOSITÓRIO 3

API — tecnologia livre

GET /comments e POST /albums com validações de status e corpo.



Todo repositório precisa de um README.md que responda:

o que o projeto testa · como instalar · como executar os testes · onde ver os relatórios e as evidências gravadas. Estrutura de pastas clara e código limpo também contam pontos.

O que será avaliado



Robot + BDD

Cenários no formato BDD bem escritos e automação funcionando.



Cypress

Login e Checkout automatizados com boas práticas e asserções reais.



Testes de API

Validação de status code e corpo das respostas GET e POST.



CI + Relatórios

Workflows orquestrados e relatórios publicados como artefatos.



Leve daqui: Capriche no README e nas evidências: é a primeira coisa que o avaliador abre — e a sua vitrine de portfólio.

Se você só lembrar de 6 coisas...

1

O desafio tem 3 partes: UI (Cypress + Robot), API e CI no GitHub Actions — um repositório para cada.

2

Escreva os cenários BDD primeiro: eles são roteiro do teste manual, da gravação e da automação.

3

Execute manualmente e grave a evidência de cada cenário antes de automatizar.

4

Valide sempre o “Então”: mensagem na tela, status code 200/201, corpo da resposta.

5

Pipeline: rodar a cada push/PR e publicar relatórios como artefatos.

6

README claro + evidências organizadas = pontos extras e portfólio pronto.

Palavras desta aula

BDD	Desenvolvimento orientado a comportamento: descreve o sistema por histórias (Dado/Quando/Então).
Gherkin	A linguagem dos cenários BDD, legível por pessoas e por ferramentas como o Robot.
Endpoint	A “porta” da API: o endereço de um recurso, como /comments ou /albums.
Status code	O código da resposta HTTP: 200 (OK), 201 (criado), 404 (não achou), 500 (erro no servidor).
Pipeline / Workflow	Sequência automática de passos no GitHub Actions que roda seus testes a cada push.
Artefato	Arquivo gerado pelo pipeline (relatórios, screenshots) que fica disponível para download.
Evidência	Prova de que o teste foi executado: gravação de tela, screenshot, log ou relatório.
Smoke test	Bateria rápida que confere se o essencial funciona antes de testar fundo.

Para estudar e consultar



SauceDemo — o e-commerce do desafio

<https://www.saucedemo.com>



JSONPlaceholder — a API fake do desafio

<https://jsonplaceholder.typicode.com>



Vídeo de referência do fluxo de testes

<https://youtu.be/T68oJWTKoIA>



Documentação do Cypress

<https://docs.cypress.io>



Robot Framework + SeleniumLibrary

<https://robotframework.org>



GitHub Actions — documentação oficial

<https://docs.github.com/actions>



Syllabus CTFL 4.0 (ISTQB/BSTQB)

<https://bstqb.org.br>